

A3 and LR(0) - Results



Attempt 1 of 1

Written Feb 26, 2026 10:23 AM - Feb 26, 2026 11:21 AM

Attempt Score **12 / 12 - 100 %**

Overall Grade (Highest Attempt) **12 / 12 - 100 %**

Question 1

1 / 1 point

In A3, In what order must you run the build commands?

- ☐ javac → java JLex.Main → java java_cup.Main
- ☐ java java_cup.Main → java JLex.Main → javac
- ✓ ☒ java JLex.Main → java java_cup.Main → javac
- ☐ javac → java A3User → java JLex.Main

Question 2

1 / 1 point

in A3, Why can't you compile calc.lex.java immediately after running JLex?

- ☐ JLex has a bug that produces broken Java
- ✓ ☒ calc.lex.java references CalcSymbol, which hasn't been generated yet

- ☐ You need to rename it to A3Scanner first
- ☐ The JDK version is incompatible

Question 3

1 / 1 point

Is BEGIN END a valid Block in the TINY grammar?

- ☐ Yes, empty blocks are always allowed
- ✓ ☒ No, because

Block → BEGIN Statement+ END

requires at least one statement

- ☐ Yes, if it's inside a method
- ☐ No, because BEGIN must be followed by a semicolon

Question 4

1 / 1 point

Which identifier definition is correct for TINY?

- ☐ [a-zA-Z_][a-zA-Z0-9_]* – letters, digits, underscores
- ✓ ☒ [a-zA-Z][a-zA-Z0-9]* – letters and digits only, no underscore
- ☐ [0-9][a-zA-Z]* – starts with a digit

- ☐ [a-z][a-zA-Z0-9_]* – lowercase start only

Question 5

1 / 1 point

When renaming calc to A3, which of the following changes is NOT required?

- ☐ Renaming the file calc.lex to A3.lex
- ☐ Changing CalcScanner to A3Scanner inside the lex file
- ☐ Changing CalcSymbol to A3Symbol inside the lex file
- ✓ ☒ Changing the JDK installation folder name

Question 6

1 / 1 point

What should happen when the scanner encounters an unknown symbol like @?

- ☐ Silently discard it with {}
- ☐ Print an error message to stdout and continue
- ✓ ☒ Return new Symbol(A3Symbol.error) so the parser rejects the program
- ☐ Throw a Java exception immediately

Question 7

1 / 1 point

Why do we need to remove the RESULT assignment in most rules of A3.cup ?

- ☐ It makes the parser faster by skipping evaluation
- ✓ ☒ It changes the parser from evaluating expressions to only recognizing syntax
- ☐ It removes the need for terminal declarations
- ☐ It prevents the parser from building a parse tree

Question 8

1 / 1 point

What is wrong with this rule in A3.cup?

`assignStmt ::= ID ASSIGN expr`

- ☐ ID should be lowercase
- ☐ ASSIGN is not a valid terminal name
- ✓ ☒ The rule is missing a semicolon at the end
- ☐ expr must be wrapped in parentheses

Question 9

1 / 1 point

LR parsers are classified as

- ☐ Top-down parsers using leftmost derivation
- ✓ ☒ Bottom-up parsers using rightmost derivation in reverse

- ☐ Top-down parsers using rightmost derivation
- ☐ Bottom-up parsers using leftmost derivation

Question 10

1 / 1 point

The stack of an LR parser stores

- ☐ Only terminals
- ☐ Only nonterminals
- ☒ States (and usually grammar symbols interleaved)
- ☐ Only input symbols

Question 11

1 / 1 point

In an LR parsing table, the ACTION part is indexed by

- ☐ (state, nonterminal)
- ☒ (state, terminal)
- ☐ (nonterminal, terminal)
- ☐ (terminal, state)

Question 12

1 / 1 point

In an LR parsing table, the GOTO part is indexed by

- ☐ (state, terminal)
- ✓ ☒ (state, nonterminal)
- ☐ (nonterminal, terminal)
- ☐ (nonterminal, state)

Done